

Purpose:

Introduce the topic and set expectations for the presentation.

How to explain it:

This presentation is a practical guide for data engineering system design. The goal is not to memorize one architecture. The goal is to learn a repeatable way to take an unclear business problem and turn it into a design that is scalable, reliable, and understandable.

Tell the audience that the deck moves from requirements to architecture, data modeling, storage, quality, and operations. It also includes an end-to-end food-delivery example and working Spark/Delta Lake patterns.

Key takeaway:

Good system design begins with understanding the problem. Technology choices come later.

AI ERP relevance:

For an AI ERP company, this framework connects systems of record such as finance, HR, procurement, and inventory to analytics, forecasting, copilots, and controlled automation.

Purpose:

Explain what the learner should be able to do after using the deck.

How to explain it:

There are four main skills. First, frame the problem by asking who needs the data and how quickly they need it. Second, estimate the size of the system using simple calculations. Third, choose an architecture by comparing trade-offs instead of choosing favorite tools. Fourth, design for real production conditions such as failures, duplicates, late data, schema changes, and backfills.

Simple example:

A daily finance report and a live fraud alert need very different pipelines even if both use the same source data.

Key takeaway:

A complete answer covers both the normal data flow and how the system behaves when something goes wrong.

AI ERP relevance:

Apply the four skills to ERP by clarifying each module and persona, sizing both tenant-fleet and largest-tenant load, and treating financial correctness and tenant isolation as design invariants.

Purpose:

Show that a system design interview tests reasoning, not diagram-drawing skill.

How to explain it:

Every important architecture decision should form a clear chain. Start with a requirement, identify the constraint, make a decision, explain the trade-off, and add a control for the risk.

Use the restaurant dashboard example on the slide. The two-minute freshness requirement leads to streaming. Streaming adds state and operational complexity. Checkpoints, replayable events, monitoring, and a durable lakehouse path control those risks.

Avoid saying only, “I will use Kafka and Spark.” Explain why those components are needed and what disadvantages they introduce.

Key takeaway:

A strong design is a well-supported argument. The diagram is only a visual summary of that argument.

AI ERP relevance:

An ERP answer should explicitly connect a business requirement such as month-end finality or payroll confidentiality to architecture, trade-offs, and controls.

Purpose:

Introduce the six-step framework used throughout the deck.

How to explain it:

Work through the steps in order. Requirements tell us what success means. Pipeline design decides how data moves and is processed. Data modeling gives the information a useful structure. Storage decisions control durability, cost, and query performance. Quality and observability make the output trustworthy. Operations make the system safe to rerun, scale, and recover.

The process is not completely one-way. If a storage estimate becomes too expensive, we may revisit retention requirements. If a strict latency target is not valuable enough, we may replace streaming with cheaper batch processing.

Key takeaway:

Use the six steps as a checklist so that reliability and operations are not forgotten at the end.

AI ERP relevance:

Use the six steps for both the data platform and the AI feature: requirements, movement, semantic model, storage/index, quality/evaluation, and operations/governance.

Purpose:

Provide a simple way to manage time during a 45-minute interview.

How to explain it:

Spend the first few minutes clarifying the problem. Then perform rough sizing calculations. Draw the complete high-level data flow before spending time on details. Deep-dive into the most important components, followed by reliability and scaling. Finish by summarizing trade-offs and open assumptions.

The exact minute values are guidance, not strict rules. The important checkpoint is around minute 20: the interviewer should already understand the end-to-end flow and how it serves the users.

Simple example:

Do not spend 20 minutes choosing Kafka partitions before showing where processed data is stored and how users query it.

Key takeaway:

Breadth first, then depth. Complete the design before optimizing one small area.

AI ERP relevance:

In an ERP interview, reserve time to discuss ledger reconciliation, tenant isolation, authorization-aware AI retrieval, and human approval for write-back actions.

Purpose:

Give a reusable set of questions for requirements gathering.

How to explain it:

Ask WHO consumes the data, WHAT they need, HOW they access it, WHEN it must be available, HOW MUCH data exists, and HOW LONG it must be retained. These questions turn a vague prompt into measurable design inputs.

Write assumptions down when the interviewer does not provide an answer. For example, say, "I will assume the peak load is four times the average and events must be replayable for 30 days." This makes your reasoning visible and gives the interviewer a chance to correct the assumption.

Key takeaway:

Do not start with tools. First identify the users, outputs, service levels, scale, and constraints that the tools must satisfy.

AI ERP relevance:

Add ERP questions: which modules, legal entities, currencies, fiscal calendars, tenant tiers, user roles, and regulated fields are in scope?

Purpose:

Clarify the difference between functional and non-functional requirements.

How to explain it:

Functional requirements describe what the system produces: dashboards, tables, APIs, alerts, or ML features. Non-functional requirements describe how well the system must work: freshness, availability, correctness, security, retention, and cost.

Convert vague statements into measurable targets. “Real time” might mean under two minutes at the 95th percentile. “Reliable” might mean 99.9 percent serving availability with no silent data loss. “Accurate” might mean daily totals reconcile with the source within 0.1 percent.

Simple example:

“Show restaurant sales” is functional. “Refresh within two minutes and remain available during dinner rush” is non-functional.

Key takeaway:

Non-functional requirements usually create the strongest architecture constraints.

AI ERP relevance:

ERP non-functional requirements include balanced accounting, closed-period immutability, payroll privacy, tenant-level availability, data residency, and explainable AI outputs.

Purpose:

Demonstrate the sizing calculations expected in a data engineering design.

How to explain it:

Start with users and behavior. Five million daily users, two sessions each, and 30 events per session produce 300 million events per day. Divide by 86,400 seconds to get about 3,472 events per second on average. Apply a peak factor; at four times average, plan for about 13,900 events per second.

Next calculate storage. At 700 bytes per event, the raw input is about 210 GB per day and 76.7 TB per year. Compression may reduce this, but replication, table metadata, checkpoints, and derived layers add storage.

Key takeaway:

Show your assumptions and round sensibly. Approximate math is enough when it leads to a clear design decision.

AI ERP relevance:

Repeat the sizing at two levels: total SaaS fleet and largest tenant. Include month-end bursts, document volume, CDC record size, and vector-index growth.

Purpose:

Connect calculations to actual architecture decisions.

How to explain it:

A number is useful only when it changes the design. Peak events per second influence event-bus partitions and consumer parallelism. Daily bytes influence object storage, file compaction, and incremental processing. High-cardinality or uneven keys can create hot partitions. A two-minute SLA requires continuous processing and a fast serving store. Long retention creates lifecycle and restore requirements.

After every estimate, ask, “What does this number force me to do?” Also ask what must be measured in production, because interview assumptions can be wrong.

Key takeaway:

Translate each estimate into capacity, cost, performance, or reliability consequences. Do not stop after writing the arithmetic.

AI ERP relevance:

For ERP, estimates should influence tenant quotas, hot-ledger partitioning, close-period capacity, cache isolation, index sharding, and per-tenant cost attribution.

Purpose:

Explain how to choose between batch and streaming.

How to explain it:

Start with the freshness requirement. If users can wait hours or a day, batch is usually simpler and cheaper. If they need updates within seconds or minutes, streaming may be required. Then check whether processing is stateful, such as windowed counts, joins, or deduplication. Stateful streaming needs checkpoints, timeouts, watermarks, and careful recovery.

Batch systems still should be incremental. They should process only new or changed partitions rather than rereading all history every time.

Simple example:

A daily executive report fits batch. A dashboard showing active deliveries during dinner rush needs streaming.

Key takeaway:

Choose the simplest processing model that meets the SLA. Streaming is valuable, but it brings additional operational responsibilities.

AI ERP relevance:

ERP often needs streaming for inventory and operations, while financial close and regulatory outputs use controlled batch finalization and reconciliation.

Purpose:

Compare batch and streaming in an easy, practical way.

How to explain it:

Batch processes a bounded set of data on a schedule. It is easier to retry and reason about because the input window can be closed. Streaming processes data continuously. It reduces latency but must handle event time, out-of-order arrival, long-running state, checkpoints, and replay.

Many real systems use both. The live path provides recent operational values, while a batch or lakehouse path produces finalized and reconciled history.

When explaining cost, note that batch compute can often turn off after work completes. Streaming infrastructure is normally always running.

Key takeaway:

Different consumers can justify different paths. A hybrid design is reasonable when it avoids forcing one SLA and cost model onto every use case.

AI ERP relevance:

Use streaming for named operational needs, but retain a durable batch/lakehouse path to restate periods and reproduce reports as of a prior close.

Purpose:

Explain three common architecture styles without treating any one as universally best.

How to explain it:

Lambda has a fast streaming path and a separate batch path. It supports low latency and accurate historical recomputation, but logic can be duplicated. Kappa uses one streaming model and replays retained events when data must be recomputed. It reduces duplicate logic but long replays and stateful processing can be difficult. A lakehouse-centered design stores durable data in object storage with an ACID table format and lets batch and streaming engines work over the same data plane.

Ask two questions: Where is the durable source of truth? How is the system replayed after a bug?

Key takeaway:

Choose an architecture based on the recovery model, operational skills, cost, and SLA—not because a pattern is fashionable.

AI ERP relevance:

A lakehouse-centered pattern is useful for shared governed ERP history, while specialized streaming, warehouse, feature, and vector stores serve distinct workloads.

Purpose:

Provide a high-level architecture that can be explained from left to right.

How to explain it:

Sources create events, database changes, or files. Ingestion safely receives and buffers the data. Processing validates, transforms, joins, and aggregates it. Storage preserves raw and curated history. Serving systems expose the result to SQL, dashboards, APIs, or machine-learning consumers.

Keep stages decoupled so one stage can slow down or fail without immediately breaking all others. Also mention the cross-cutting capabilities at the bottom: security, metadata, lineage, data quality, observability, cost controls, and deployment automation.

Key takeaway:

First show the complete flow. Then choose one or two critical stages for a deeper technical discussion.

AI ERP relevance:

In the ERP version, sources are transactional modules and documents; serving includes BI, forecasts, anomaly detection, copilots, and approval-based actions.

Purpose:

Explain the senior-level decisions behind an event-ingestion backbone.

How to explain it:

Begin with identity and ordering, not partition count. Every event needs a stable event ID and source identity so consumers can safely process duplicate delivery. Define the smallest business scope that requires order. An order may require lifecycle order; a financial ledger may require tenant plus legal entity plus ledger order. Global ordering is usually unnecessary and limits scale.

Choose a partition key that preserves the required local order while distributing peak traffic. Measure the largest key and tenant because fleet averages can hide a hot partition. If a key is too hot, sub-shard only when the business can tolerate independent order across shards. Size the event service using measured bytes and records per partition, consumer capacity, desired backlog-drain time, replay retention, and service quotas.

For AI ERP, the envelope must carry tenant, aggregate, source transaction or LSN, occurred and produced time, schema version, and trace identity. Downstream files and state should retain that lineage.

Key takeaway:

Partitioning encodes ordering, parallelism, tenant fairness, and recovery. It cannot be selected from throughput alone.

Purpose:

Explain how to select serving technologies for BI, applications, machine learning, and ERP copilots.

How to explain it:

Start with query shape and latency. BI and finance use scans, joins, governed measures, and concurrency, so a warehouse, lakehouse SQL engine, or OLAP system may fit. Operational dashboards need fast recent aggregates and controlled updates. Application APIs need predictable point or range access. Machine learning needs reproducible offline features and sometimes low-latency online values. An ERP copilot needs authorized structured queries plus permission-filtered document retrieval.

Every serving copy needs a refresh and correction method, freshness indicator, authorization policy, reconciliation with durable truth, and fallback when stale or unavailable. Cache keys must include tenant and entitlement context. Add a new store only when its access pattern justifies the synchronization and operating cost.

Key takeaway:

Share governed data and semantics, but choose serving systems by access pattern. One physical store rarely satisfies every workload well.

Purpose:

Explain Bronze, Silver, and Gold as clear data contracts.

How to explain it:

Bronze preserves source data and ingestion metadata. Its main purpose is recoverability, so avoid destructive cleaning here. Silver converts raw data into trusted, reusable entities by parsing, casting, deduplicating, and enforcing important rules. Gold provides business-ready products such as stable metrics, aggregates, and consumer-specific tables.

A record that cannot be parsed should normally be retained or quarantined rather than silently dropped. Gold changes should be carefully versioned because dashboards and other consumers may depend on exact names and meanings.

Key takeaway:

Bronze lets us rebuild. Silver gives shared truth. Gold gives consumers an understandable and stable business interface.

AI ERP relevance:

ERP Bronze should preserve CDC order and tenant identity; Silver should create canonical parties, transactions, resources, and ledgers; Gold should expose governed measures.

Purpose:

Show how to begin dimensional modeling with the table grain.

How to explain it:

The grain states exactly what one row represents. In this example, one fact-table row represents one lifecycle event for one order. Measures such as amount or duration belong in the fact. Descriptive context, such as customer segment, restaurant cuisine, date, and location, belongs in dimensions.

After defining grain, identify keys, dimensions, and measures. Then discuss late-arriving facts and dimension history. A fact must join to the dimension version that was valid when the event happened, not necessarily the current version.

Key takeaway:

Always state the grain before listing columns. An unclear grain causes duplicates, incorrect totals, and confusing joins.

AI ERP relevance:

State ERP grain precisely: one journal line, invoice line, inventory movement, employee event, or order status change. Mixing grains creates incorrect balances.

Purpose:

Compare common analytical modeling choices.

How to explain it:

A star schema is a strong default for governed business intelligence because facts and dimensions make grain and reuse clear. A snowflake normalizes dimensions further, reducing duplication but adding joins and user complexity. A one-big-table model can simplify a specific query path but duplicates data and becomes harder to update consistently. Data Vault is useful when auditability and source-history integration are major goals, but business-facing marts are still normally required.

State both your default and the condition that would make you switch.

Simple example:

Use a star for shared sales reporting. Build a carefully managed wide table only if a proven high-volume dashboard benefits from avoiding repeated joins.

Key takeaway:

The model should match consumer access patterns and governance needs.

AI ERP relevance:

A star schema is strong for finance and operations BI; a canonical integration model can feed it; wide tables should be limited to proven tenant-safe serving paths.

Purpose:

Explain how dimension changes can be stored over time.

How to explain it:

SCD Type 1 overwrites the old value. Use it when history is not needed, such as correcting a spelling error. SCD Type 2 inserts a new version and keeps effective start and end dates. Use it when historical reporting must show the value that was true at that time. SCD Type 3 stores only limited previous information in extra columns.

In the sample table, customer C55 changed from SILVER to GOLD. The old row is closed and a new current row is added. Prevent overlapping date ranges and define how late events find the correct version.

Key takeaway:

Choose history behavior by business meaning. Not every attribute needs the same SCD strategy.

AI ERP relevance:

ERP dimensions such as employee, supplier, chart of accounts, cost center, and legal entity often require SCD2 history for correct as-of reporting.

Purpose:

Distinguish file serialization formats from analytical table formats.

How to explain it:

CSV and JSON are easy to exchange and inspect but are large and weakly typed. Avro is row-oriented and well suited to serializing individual records on event systems. Parquet and ORC are columnar, so analytical engines can read only needed columns and apply compression efficiently. Delta Lake and Iceberg are table layers over data files; they add snapshots, metadata, schema management, and transactional behavior.

A common design uses Avro or Protobuf for events and Parquet files managed by a Delta or Iceberg table for analytics.

Key takeaway:

Choose the event format for safe record exchange, the file format for efficient storage, and the table format for reliable table operations.

AI ERP relevance:

ERP CDC events may use Avro/Protobuf; analytical history may use Parquet with Delta/Iceberg; contracts and version compatibility matter across tenant customizations.

Purpose:

Explain how physical data layout affects query performance and cost.

How to explain it:

Partitioning separates data into directories or logical groups that can be skipped during a query. Date is often useful because many queries filter by time. Avoid very high-cardinality partition columns because they create too many small partitions and files. Clustering or sorting can place similar values together without a huge directory structure. Compaction combines small files, improving listing and read efficiency. Metadata maintenance removes old snapshots only after replay and time-travel requirements are considered.

Walk through the read path: partition pruning, file skipping, column pruning, and decompression.

Key takeaway:

Design layout from real filter patterns and file sizes. More partitions do not automatically mean faster queries.

AI ERP relevance:

Partition first by access and scale needs, often date plus a carefully chosen tenant/legal-entity strategy; prevent one large tenant from creating hot files or partitions.

Purpose:

Show that data quality rules should become stricter as data moves toward consumers.

How to explain it:

Bronze checks whether data can be ingested and replayed. Record parse failures and source counts, and quarantine malformed data. Silver checks whether entities are trustworthy by enforcing types, uniqueness, valid values, references, and freshness. Gold checks business meaning by reconciling metrics, validating business rules, and protecting consumer SLAs.

Every check needs an owner, threshold, and action. Decide whether failure should alert, quarantine records, stop a pipeline, or hold publication. Silent failure is the most dangerous outcome.

Key takeaway:

Quality is not one test at the end. It is a set of controls applied at the right layer with a clear response.

AI ERP relevance:

Add ERP invariants such as debit equals credit, invoice totals equal lines plus tax, valid accounting periods, inventory cannot silently disappear, and tenant IDs are complete.

Purpose:

Explain how data contracts prevent downstream surprises.

How to explain it:

A contract describes structure, meaning, service levels, ownership, and change policy. Schema includes field names, data types, and nullability. Semantics explain what the fields and events mean. SLOs define freshness and availability. Ownership identifies who approves changes and responds to incidents.

A schema registry helps enforce structural compatibility, but it is only part of a full contract. For a breaking change, publish a new version, allow old and new versions to run together, measure migration, and retire the old version only after consumers move.

Key takeaway:

Data contracts move quality checks closer to the producer, where problems are cheaper and easier to correct.

AI ERP relevance:

ERP contracts must define business meaning as well as types—for example currency units, sign conventions, fiscal period, source module, and whether an amount is posted or provisional.

Purpose:

Explain what must be monitored in a production data platform.

How to explain it:

Data telemetry measures freshness, volume, duplicates, and distribution changes. Pipeline telemetry measures job duration, event lag, watermarks, retries, and dead-letter volume. Platform telemetry measures CPU, memory, disk, shuffle, checkpoints, and service quotas. Business telemetry checks whether important numbers, such as orders or revenue, still make sense.

Alerts should describe user impact and point to an owner and runbook. Include lineage, affected partitions, recent deployments, and recovery steps so an engineer can act quickly.

Key takeaway:

Observability should answer three questions: What is wrong? Who is affected? What should we do next?

AI ERP relevance:

Monitor ERP outcomes per tenant: CDC lag, close freshness, reconciliation differences, policy denials, retrieval quality, model drift, and cost by workload.

Purpose:

Explain idempotency as more than a single write command.

How to explain it:

Idempotency means repeating the same logical work produces the same visible result. It requires a stable event or business key, deterministic transformations, an atomic write, durable progress tracking, safe retries, and reconciliation.

A message system may deliver an event more than once. That is acceptable if the target uses the event identity to deduplicate or upsert so the final table contains one correct record. Do not record source progress before the output commit succeeds, or the system may skip data after a failure.

Key takeaway:

Design the rerun path before the first run. A pipeline is not idempotent merely because one component claims exactly-once behavior.

AI ERP relevance:

ERP idempotency must cover journal postings and AI-initiated actions. Use business transaction IDs and approval/action IDs so retries cannot double-post.

Purpose:

Walk through an idempotent Delta Lake batch upsert.

How to explain it:

The code first defines a compound key using order ID and event timestamp. A window keeps only the newest source row for each key. This source deduplication matters because multiple source rows matching one target row can make a merge ambiguous or incorrect.

The MERGE updates an existing record only when the incoming updated\_at value is at least as recent as the target. Otherwise, it inserts a new record. Delta commits the change atomically, so readers do not see a partially updated table.

Mention remaining decisions: deletion behavior, null-safe keys, concurrent writers, and count reconciliation.

Key takeaway:

A safe MERGE needs a correct key, one source winner per key, deterministic update rules, and an atomic target commit.

AI ERP relevance:

For ERP MERGE keys, include tenant identity and the true business grain. Never let records from different tenants collide on a shared order or invoice ID.

Purpose:

Explain a safe method for reprocessing historical data.

How to explain it:

A backfill should use immutable Bronze input and the same transformation logic as the normal pipeline. Select a bounded date range, rebuild the Silver output, and validate row counts, checksums, and business totals. Then use `replaceWhere` to atomically replace only that date range.

Keep the run ID, code version, date range, and validation results for audit. Ensure every output record satisfies the replacement predicate so records are not accidentally written outside the intended range. Coordinate downstream refreshes because replacing history can change dashboards and aggregates.

Key takeaway:

Backfills are planned production operations, not manual one-off scripts. Scope, validate, publish atomically, and record what changed.

AI ERP relevance:

ERP backfills should respect closed periods, tenant maintenance windows, regulatory retention, downstream consolidations, and a formal restatement/audit process.

Purpose:

Explain how to make a Spark Structured Streaming output idempotent.

How to explain it:

The stream uses a durable checkpoint to track source progress and calls `foreachBatch` for each micro-batch. Inside the function, records are deduplicated and merged using a stable event ID. If the batch is attempted again, the deterministic MERGE produces the same table outcome.

The note on the right corrects an important API issue. Transaction options such as `txnAppld` and `txnVersion` are not normally chained onto `DeltaMergeBuilder`. For MERGE, rely on checkpointed input progress and idempotent merge logic. A batch ID can also be recorded in an audit ledger.

Key takeaway:

A checkpoint controls progress. The merge key and update rules control the final table result. Both are required.

AI ERP relevance:

A streaming ERP pipeline should preserve source transaction order, checkpoint offsets, deduplicate by tenant plus source identity, and reconcile against module control totals.

Purpose:

Present a safe strategy for handling changing source schemas.

How to explain it:

Bronze should be resilient. Keep the raw payload, schema identifier, ingest timestamp, and unexpected fields so data is not lost. Silver should be explicit: cast expected types, apply defaults, quarantine invalid records, and test compatibility. Gold should be stable because dashboards and applications depend on its names and meanings.

Additive nullable fields are often backward-compatible. Renames, removals, type changes, and semantic changes normally require versioning and migration. Even when a platform can merge schemas automatically, the organization still needs a change policy.

Key takeaway:

Be permissive at the edge so data survives, but strict at the trusted and business-facing layers so consumers are protected.

AI ERP relevance:

Tenant custom fields make schema governance critical: keep a stable core, capture extensions safely, and promote only approved fields into shared semantic products.

Purpose:

Show practical code for controlled schema evolution.

How to explain it:

The Bronze example uses a rescued-data column to capture fields that do not match the known schema. This prevents unexpected data from disappearing and allows the team to investigate drift. The Silver example first validates required columns and compatible types. Only then does it enable mergeSchema for an approved additive change.

Avoid enabling global automatic schema merge as a convenience. It may allow surprising or weakly related data into important tables. Use a production gate: compatibility test, owner approval, migration plan, consumer validation, rollout, and drift monitoring.

Key takeaway:

Automatic schema capability is a tool, not a governance policy. Validate and approve changes before trusted layers evolve.

AI ERP relevance:

Use rescued data for observation, not as a permanent substitute for contracts. ERP schema changes should pass module-owner, data, security, and downstream compatibility checks.

Purpose:

Explain how the system handles failures, disaster recovery, and major growth.

How to explain it:

For each failure, name four things: the failure domain, detection signal, recovery action, and possible data-loss boundary. Consumer crashes need durable checkpoints. Poison records need quarantine and replay. Regional outages need replicated data and tested failover. Ten-times traffic needs partition and consumer scaling plus hot-key analysis. Bad deployments need versioning and rollback. Corrupt output needs snapshots and rebuild from Bronze.

RPO is the maximum acceptable data loss. RTO is the maximum acceptable recovery time. Product features do not prove these targets; recovery drills do.

Key takeaway:

A reliable design includes a tested recovery story for the most important failure modes.

AI ERP relevance:

ERP disaster recovery should classify source ledgers and audit evidence as higher priority than rebuildable dashboards, features, embeddings, and predictions.

Purpose:

Introduce the end-to-end food-delivery design problem and quantify it.

How to explain it:

There are two consumers. Restaurant partners need operational metrics in under two minutes. Executives need accurate daily history. Five million orders with six lifecycle events produce 30 million events per day. This is about 350 events per second on average, with a supplied peak of about 833 events per second. Raw data is roughly 30 GB per day at one KB per event.

Challenge assumptions politely. The stated peak is only about 2.4 times the average, so promotions, regional concentration, retries, and future growth may require more headroom.

Key takeaway:

Different SLAs justify different serving paths, while durable raw data provides one foundation for replay and reconciliation.

AI ERP relevance:

Treat the food-delivery example as an operational ERP analogy: replace restaurants with tenants or business units and orders with invoices, shipments, or work orders.

Purpose:

Walk through the proposed food-delivery architecture from source to consumer.

How to explain it:

Order services publish lifecycle events to Kafka using order ID as the key to preserve per-order ordering. The streaming path parses, validates, deduplicates, and calculates recent metrics. It writes live values to Redis or a low-latency OLAP store for restaurant dashboards.

At the same time, raw events land in Bronze. Silver creates clean, conformed entities, and Gold creates finalized business metrics for executive reporting. The lakehouse path provides history, backfills, and reconciliation when late events revise live values.

Mention the cross-cutting controls: contracts, catalog, lineage, quality, monitoring, checkpoints, replay, and access control.

Key takeaway:

The live path serves speed; the lakehouse path provides durability, correctness, and historical reproducibility.

AI ERP relevance:

For AI ERP, extend the architecture with tenant policy enforcement, canonical business semantics, feature/model lineage, and permission-filtered document retrieval.

Purpose:

Demonstrate that the design handles more than the happy path.

How to explain it:

Trace six scenarios. A normal event flows through validation, streaming, serving, and Bronze archival. A late event uses event time and may revise a recent window. Duplicate delivery is controlled by the event key and idempotent MERGE. A bad schema is blocked or quarantined. A backfill rebuilds bounded history from Bronze and publishes atomically. A ten-times traffic spike is handled with more partitions and consumers, while stale-but-labeled data can provide graceful degradation.

Use one event as a test: can it be traced, replayed, and explained without creating a duplicate?

Key takeaway:

A credible architecture explains normal flow, late data, duplicates, failure, replay, and scaling.

AI ERP relevance:

Add ERP failure scenarios: late journal corrections, reopened periods, tenant key rotation, role changes during cached AI sessions, and partial multi-module transactions.

Purpose:

Give the presenter a simple sentence pattern for explaining decisions.

How to explain it:

For every major choice, say: Because of this requirement, I choose this component or pattern. This provides a benefit. It also creates a cost or risk. I control that risk with a specific mitigation.

Use the worked example. A two-minute SLA leads to streaming. Streaming gives continuous updates but adds state and always-on operations. Bounded windows, checkpoints, lag alerts, retained events, and lakehouse reconciliation reduce those risks.

This format prevents a tool-list answer and makes trade-offs visible. Also say what is intentionally out of scope until the interviewer asks.

Key takeaway:

Requirement, decision, benefit, cost, mitigation—repeat this structure throughout the interview.

AI ERP relevance:

A strong ERP narration begins with invariants such as tenant isolation and financial balance before naming Kafka, Spark, a lakehouse, a vector store, or an LLM.

Purpose:

Provide a final completeness check for a design answer.

How to explain it:

Review six categories. Requirements should name users, outputs, measurable SLOs, retention, and security. Scale should cover average and peak throughput, storage, growth, and skew. Architecture should show the complete flow and explain batch or streaming. Data design should state grain, keys, model, format, and layout. Reliability should cover quality, replay, idempotency, backfills, schema change, and disaster recovery. Trade-offs should include cost, complexity, bottlenecks, and alternatives.

If an item is unknown, do not hide it. State it as an assumption or open question and describe how you would validate it.

Key takeaway:

A transparent gap is better than an unsupported claim.

AI ERP relevance:

Use this scorecard per ERP workload and per AI use case; a platform can meet dashboard SLAs while failing authorization, auditability, or financial finality.

Purpose:

Offer a one-page template that can be copied onto a whiteboard.

How to explain it:

Create six areas before solving the prompt. Write users and outputs first. Add service levels and constraints. Perform scale calculations. Draw the complete data flow. Define data grain, keys, models, formats, and layout. Finally, add operational controls such as quality, monitoring, replay, backfills, schema evolution, and disaster recovery.

At the end, identify the likely bottleneck at ten-times scale, the largest correctness risk, the largest cost driver, an alternative architecture, and any unresolved assumptions.

Key takeaway:

The template keeps the discussion organized and prevents important production topics from being forgotten.

AI ERP relevance:

Add three boxes to the template when needed: tenant-isolation model, business invariants, and AI authorization/evaluation/write-back controls.

Purpose:

Summarize the most important lessons from the presentation.

How to explain it:

Begin with users and measurable service levels. Estimate peak load and storage before choosing technology. Prefer the simplest architecture that satisfies the requirement. Define data grain, history, schema contracts, and access patterns. Plan reruns, replay, backfills, and migrations as normal system behavior. Explain every trade-off using a requirement and a mitigation.

Remind the audience that complexity is not a sign of quality. A smaller batch architecture can be the best answer for a daily report. A streaming architecture is justified only when its lower latency creates real business value.

Key takeaway:

The best design is understandable, measurable, recoverable, and no more complicated than necessary.

AI ERP relevance:

For AI ERP, the simplest acceptable design must still preserve transactional truth, tenant boundaries, semantic consistency, evidence, and safe human-controlled actions.

Purpose:

Explain why an AI-powered ERP platform is a harder design problem than a standard analytics pipeline.

How to explain it:

ERP is a system of record. Incorrect or missing transactions can change financial statements, payroll, tax, inventory, or customer commitments. It is also multi-tenant SaaS, so one customer's data must never appear in another customer's query, cache, model context, vector result, trace, or support workflow. Operational users need fresh data, while auditors need reproducible history. AI copilots add retrieval, model, evaluation, and action-control requirements.

Describe the four pillars together. Transactional truth supplies trusted facts. Tenant isolation controls every data path. Operational analytics provides timely decisions. AI copilots use governed context and should cite evidence or abstain.

Key takeaway:

An AI ERP design is not just a lakehouse plus an LLM. It is a governed connection from authoritative transactions to permission-aware decisions and actions.

Purpose:

Show that an AI ERP platform contains several workloads with different service levels.

How to explain it:

Financial close values correctness, restatement, and audit evidence more than second-level latency. Operations dashboards need low CDC lag and predictable serving performance. Forecasting requires reproducible point-in-time features rather than only current values. A copilot needs interactive response time, but it also needs authorized, fresh, cited evidence. Audit workloads may run infrequently but require long retention and immutable lineage.

Do not put every workload into one database. Use shared canonical data, governance, tenant identity, and metrics, then choose a warehouse, operational OLAP store, feature store, vector index, or archive according to the access pattern.

Key takeaway:

Shared truth does not require one serving technology. Match each workload to its latency, consistency, query, and isolation needs.

Purpose:

Present an end-to-end reference architecture for ERP data, analytics, machine learning, and copilots.

How to explain it:

Transactional ERP modules emit CDC records and domain events with source offsets, transaction IDs, schemas, and tenant identity. Documents use a separate OCR and parsing path. Bronze preserves original changes and files. Silver maps module-specific records into canonical ERP entities. Gold supplies governed measures and features.

The semantic layer serves finance and operations BI. The feature store supports reproducible model training and scoring. The vector index contains document chunks and metadata with mandatory tenant and ACL filters. An AI gateway performs authorization, retrieval, prompt controls, citation, caching, and audit before the copilot responds or proposes an action.

Cross-cutting controls must follow the data across every plane.

Key takeaway:

Keep transactional authority, analytical truth, and AI serving separate, but connect them with identity, semantics, lineage, and policy.

Purpose:

Explain the main multi-tenant storage and compute isolation choices.

How to explain it:

Shared tables are efficient for many small tenants, but every key and query must include tenant identity and policy enforcement must be extensively tested. A schema or database per tenant offers clearer restoration and logical boundaries, but migrations and metadata become expensive at large scale. A dedicated stack provides strongest separation for regulated or very large customers, at higher cost. Many ERP companies use a tiered hybrid and route tenants by size, risk, geography, or contract.

Isolation also applies outside tables. Include encryption keys, object paths, checkpoints, caches, temporary files, model features, prompt traces, vector namespaces, observability tags, and support access. Add per-tenant quotas and fair scheduling to control noisy neighbors.

Key takeaway:

Multi-tenancy is not a WHERE clause. Tenant identity and policy must be impossible to omit at every boundary.

Purpose:

Explain how CDC becomes financially correct analytical data.

How to explain it:

The database transaction log provides ordered technical changes, but financial meaning comes from business transactions and accounting rules. Capture source transaction ID, row identity, operation, LSN or offset, commit time, tenant, legal entity, and schema version. Preserve raw before/after images. Build canonical ledger lines by deduplicating and applying changes in source order.

Create an atomic relationship between consumed source range and target commit. A retry must merge to the same result. Then validate accounting invariants, such as debit equals credit, and reconcile source counts and control totals. Corrections to closed periods should be explicit reversals or adjustment versions, not destructive overwrites.

Key takeaway:

Exactly-once reporting is proved by identity, ordering, idempotent writes, atomic progress, invariants, and reconciliation—not by the CDC tool alone.

Purpose:

Connect a canonical ERP model to secure AI retrieval.

How to explain it:

ERP modules use different names for similar concepts. A canonical model gives stable identities and relationships for parties, legal entities, transactions, resources, and accounting. A governed semantic layer defines measures such as booked revenue, available inventory, or overdue receivables consistently.

For an AI question, first establish tenant, user, role, purpose, and current entitlements. Retrieve structured facts through governed SQL and unstructured evidence through a vector or search index. Apply row, field, document, and effective-time policy before context reaches the model. Include source, time, currency, and freshness metadata. The answer should cite evidence and abstain when the available context is incomplete.

Key takeaway:

The copilot should inherit ERP semantics and authorization. Retrieval must not create a second, weaker security model.

Purpose:

Show how data governance extends across the complete AI lifecycle.

How to explain it:

At ingest, classify data, validate contracts, and record consent or purpose restrictions. During curation, preserve lineage and construct point-in-time-correct data. During training or indexing, record dataset snapshots, feature definitions, model or embedding versions, parameters, and evaluation sets. At serving, record tenant policy, retrieved evidence, prompt template, model, cache behavior, and response. Evaluate groundedness, business accuracy, fairness, drift, and abstention.

For write-back, treat the LLM as an untrusted planner. Convert the proposal into a typed command, validate ERP rules, require human approval according to risk, attach an idempotency key, and log before/after state. Support rollback or compensating action.

Key takeaway:

An AI ERP company must trace an answer or action from source evidence through policy and model version to the final outcome.

Purpose:

Demonstrate tenant-aware sizing and resilience planning for an ERP SaaS fleet.

How to explain it:

The example assumes 8,000 tenants generating an average of 250,000 changes each day, or about two billion changes. This is roughly 23,000 changes per second on average. A six-times month-end peak requires about 139,000 changes per second before extra headroom. At 1.2 KB per change, raw volume is around 2.4 TB per day.

Fleet average is not enough. One large tenant or legal entity can dominate a partition during close. Plan buffering, pre-scaling, tenant quotas, weighted scheduling, and hot-key splitting. Define separate recovery tiers: source ledgers and audit evidence need stronger RPO/RTO than rebuildable aggregates, features, embeddings, and predictions.

Key takeaway:

Size for total fleet, peak calendar events, and largest tenant. Protect critical truth more strongly than derived AI assets.

Purpose:

Give a repeatable method for answering an AI ERP system-design question.

How to explain it:

First clarify modules and users. Then state non-negotiable invariants: tenant isolation, balanced accounting, as-of correctness, authorization, and audit. Define service levels for CDC, report finality, AI response time, and recovery. Size both fleet and largest tenant. Draw three connected planes: transactional, analytical, and AI serving. Explain the canonical model and metric semantics. Prove replay, reconciliation, backfill, and schema migration. Finally prove AI authorization, grounding, evaluation, approval, idempotent action, and audit.

Close by naming the hardest trade-off. For example, low latency may expose provisional values, so label them and run a finalized reconciled path.

Key takeaway:

Lead with business invariants and risks, then justify technology as a way to satisfy them.

Purpose:

Provide deeper model answers for complex questions 1 through 4.

1. Near-real-time analytics for 10,000 tenants:

Clarify tenant sizes, modules, regions, freshness, and query patterns. Capture CDC into a partitioned durable log carrying tenant and source identity. Land immutable Bronze, create canonical Silver, and publish tenant-aware Gold. Use a low-latency OLAP store for live operations and a warehouse/lakehouse for governed history. Enforce row/field policy, per-tenant quotas, lag SLOs, cost attribution, and dedicated tiers for regulated or very large customers. Test noisy-neighbor and cross-tenant access failures.

2. Exactly-once financial reporting:

Preserve source transaction ID, row ID, LSN, commit order, tenant, and legal entity. Deduplicate and MERGE using source identity; commit consumed ranges with target versions. Do not publish a period until debit/credit and source control totals reconcile. Keep correction and restatement lineage. Explain that transport may be at-least-once while the business result is idempotent and reconciled.

3. Tenant custom fields:

Keep a stable shared core. Capture extensions in a typed key/value, JSON/variant, or tenant extension structure with schema metadata. Validate per-tenant contracts and index only commonly queried fields. Promote a custom field to a governed column only after ownership, type, meaning, and usage justify it. Prevent per-tenant columns from exploding shared-table schemas.

4. Closed-period corrections:

Do not overwrite history. Post a reversal or adjustment, keep effective and recorded times, version as-of facts, and rebuild only affected tenant/entity/period aggregates. Require approval, record reason and actor, preserve old report versions, and notify downstream consumers of restatement.

Key takeaway:

Strong ERP answers combine tenant boundaries, source identity, business invariants, controlled extensibility, and auditable correction.

Purpose:

Provide deeper model answers for complex questions 5 through 8.

5. ERP copilot over tables and documents:

Use an AI gateway. Authenticate tenant/user/purpose, resolve entitlements, classify the question, and retrieve structured facts through the semantic layer plus document chunks through hybrid search. Filter before model context, include freshness and citations, and prefer deterministic tools for calculations. The model should abstain or ask for clarification when evidence conflicts. Log evidence IDs and model/prompt versions.

6. Prevent cross-tenant RAG leakage:

Use physically separate indexes for high-risk tenants or mandatory tenant namespaces and filters for shared indexes. Copy ACLs and effective dates to every chunk. Encrypt, use tenant-aware caches, and never trust an LLM to enforce policy. Add policy-as-code, negative authorization tests, adversarial prompts, canaries, redacted traces, and incident kill switches.

7. Forecast features without leakage:

Define event time and availability time. Build training rows using point-in-time joins so no feature uses information known after the prediction timestamp. Version feature code, source snapshots, labels, and fiscal calendars. Compare offline and online feature values, monitor skew, and support reproducible retraining by tenant cohort.

8. Trustworthy AI evaluation:

Measure retrieval recall, citation support, groundedness, calculation/business-rule accuracy, freshness, permission correctness, abstention, latency, and cost. Evaluate by module, tenant tier, language, role, and risk. Use curated test cases plus production feedback and shadow evaluation. A fluent answer with unsupported numbers must fail.

Key takeaway:

AI quality in ERP includes authorization and business correctness, not only natural-language quality.

Purpose:

Provide deeper model answers for complex questions 9 through 12.

9. Privacy deletion with audit retention:

Classify personal fields and identify legal basis per product and region. Separate business transaction evidence from direct identity where possible and tokenize or pseudonymize identifiers. Delete or tombstone permitted data in source, lakehouse, warehouse, feature store, vector index, caches, logs, and training queues. Preserve only legally required evidence under restricted purpose and legal hold. Maintain a deletion manifest and verify completion.

10. Two-year backfill without live impact:

Read immutable snapshots using a versioned transformation. Run isolated compute with quotas and lower-priority scheduling. Break work into resumable tenant/date manifests, handle skewed tenants separately, and publish partitions incrementally after quality and financial control-total validation. Record code/input/output versions and coordinate restatements. Throttle if CDC lag or serving SLOs degrade.

11. Ten-times month-end spike:

Pre-scale based on fiscal calendars, retain enough log capacity, increase consumers, and remove tenant/legal-entity hot keys. Apply weighted fair scheduling and prioritize ledger ingestion and close-critical outputs over secondary AI indexing. Use backpressure and display labeled stale analytics rather than dropping data. Load-test with the largest tenant and correlated closes.

12. Regional outage with residency:

Classify data by residency and criticality. Replicate only allowed data to an approved paired region; replicate code, schemas, catalog, policies, keys, and checkpoints appropriately. Define RPO/RTO separately for transactions, analytics, and rebuildable AI indexes. Use DNS/routing and controlled failover, reconcile after recovery, and exercise the runbook regularly.

Key takeaway:

Privacy, backfills, peak events, and disaster recovery require explicit scope, prioritization, audit evidence, and tested operational procedures.

Purpose:

Frame the interview around Campfire's stated mission and values from the candidate-guide summary.

How to explain it:

Campfire is described as an AI-native ERP that aims to give accounting and finance teams superpowers by reducing repetitive work and providing timely financial insight. Translate that mission into engineering priorities: trustworthy financial data, automation that users can understand and control, responsive customer experiences, and fast iteration without compromising accounting correctness.

In the conversation, demonstrate transparent accountability by stating assumptions and risks directly. Show customer-centric innovation by beginning with an accountant's workflow and measurable outcome. Show low ego and growth mindset by comparing alternatives and changing direction when evidence is better. Show quality with velocity through small releases, automated gates, canaries, and rollback. Show collaborative excellence by including accounting, product, security, and engineering ownership.

Key takeaway:

Do not merely repeat company values. Make them visible in how you clarify, design, communicate trade-offs, and discuss mistakes.

Purpose:

Connect data engineering capabilities to practical customer outcomes for finance teams.

How to explain it:

Faster close requires timely ingestion, clear exception queues, reconciliation, reproducible period snapshots, and restatement history. Reducing manual work requires safe document extraction, matching, coding recommendations, approval workflows, and idempotent actions. Real-time intelligence requires governed measures with freshness and finality indicators. Trusted AI requires permission-aware retrieval, citations, evaluation, and human control.

For each capability, name the user and pain before the technology. For example: a controller spends hours finding an unexplained variance; lineage and exception prioritization reduce investigation time. Explain the financial invariant that cannot be sacrificed and the smallest safe release that proves value.

Key takeaway:

The strongest Campfire answer turns architecture into an outcome: fewer manual steps, faster close, better explanations, or higher financial confidence.

Purpose:

Show a reference architecture for AI-native financial intelligence and automation.

How to explain it:

Financial systems emit ordered CDC and domain events; documents use OCR and extraction. Bronze retains immutable evidence. Silver creates canonical finance entities. Gold exposes governed metrics. An audit ledger records versions, approvals, and actions.

The semantic layer ensures reports and AI use the same definitions. Features and models support forecasting and anomaly detection. Secure retrieval combines authorized structured data and documents. User experiences include close workflows, intelligence, and a copilot. Any proposed write-back becomes a typed command that passes accounting rules, authorization, approval, idempotency, and audit before posting.

Follow tenant identity, source transaction, time, schema, and policy through every arrow. Explain provisional versus finalized outputs and how corrections are replayed.

Key takeaway:

Transactional authority, analytical truth, and AI automation remain separate responsibilities connected by semantics, lineage, policy, and reconciliation.

Purpose:

Make the non-negotiable controls explicit before discussing speed or convenience.

How to explain it:

Financial controls prove that journals balance, source totals reconcile, closed periods are not silently rewritten, and every published measure has a cutoff and lineage. Tenant controls ensure customer identity and authorization are present in tables, object paths, caches, indexes, prompts, traces, and support tools. AI controls treat the model as an untrusted planner: output is typed, validated, approved according to risk, idempotent, and audited.

Quality with velocity means automating these controls. A canary tenant, contract tests, reconciliation gates, feature flags, and quick rollback let the team ship faster because risk is visible and bounded.

Key takeaway:

Move quickly by making critical controls automatic and repeatable, not by bypassing them.

Purpose:

Provide a Campfire-relevant system-design prompt and a clear answer structure.

How to explain it:

Clarify controllers and accountants, live exceptions versus finalized close, legal entities, currencies, fiscal calendars, and restatement behavior. Estimate tenants, largest tenant, journal lines, month-end concentration, document volume, and retention. Draw ordered CDC and document ingestion into immutable evidence, canonical finance models, reconciled metrics, and authorized experiences.

Prove tenant isolation, balanced accounting, source reconciliation, idempotent replay, versioned close snapshots, and cited AI explanation. Add per-tenant lag, exception telemetry, canary rollout, audited backfills, RPO/RTO, and cost attribution.

End with the main tension: users benefit from fast provisional intelligence, while finance requires reproducible finality. The design supports both and labels the difference.

Key takeaway:

Lead with financial invariants and customer workflow, then justify pipeline and AI components as mechanisms to meet them.

Purpose:

Turn Campfire's values into concrete evidence a candidate can show.

How to explain it:

Transparent accountability appears when you own an incident, communicate scope, and add prevention. Customer-centric innovation appears when user research changes priorities and produces a measurable finance outcome. Growth mindset and low ego appear when you invite criticism and adopt a better idea. Quality with velocity appears through small slices, automated tests, reconciliation, canaries, and rollback. Collaborative excellence appears through aligned domain, product, security, and engineering decisions.

A good story includes customer stakes, your specific role, alternatives, action, measurable result, an honest difficulty or mistake, learning, and the durable change afterward. Share team credit while being clear about your contribution.

Key takeaway:

Values answers are strongest when they contain a decision and evidence, not adjectives about yourself.

Purpose:

Prepare the candidate for both the practical interview setup and a collaborative discussion.

How to explain it:

Prepare three stories: customer empathy and innovation; quality with velocity; transparent accountability and collaboration. Include metrics and lessons. During technical questions, pause, clarify, state invariants, estimate, compare alternatives, admit uncertainty, invite correction, and summarize recovery.

Follow the supplied logistical guidance: test camera, microphone, and internet; use smart casual clothing that feels natural; and be ready for virtual and multi-hour onsite conversations. Ask questions that reveal real customer pain, finality rules, AI action boundaries, scaling pressure, engineering impact, and collaboration with accounting experts.

The goal is not to perform a memorized answer. Treat the interview as a design session with shared problem solving.

Key takeaway:

Preparation creates confidence, but curiosity, directness, and collaboration should remain visible in the conversation.

Purpose:

Identify the source and explain how to use the material responsibly.

How to explain it:

The deck is based on the user-provided summary of Afaque Ahmad's video and expands it into a reusable teaching and interview framework. The technology names are examples, not fixed recommendations. Exact APIs, feature support, and operational behavior can vary by platform and version, so implementation details should be checked in the target environment.

The streaming idempotency section intentionally corrects the supplied example that attached transaction options to a Delta MERGE builder. The deck uses checkpointed progress plus deterministic MERGE logic as the main pattern.

Key takeaway:

Use the framework consistently, but replace assumptions and technologies according to the real users, cloud, team, compliance rules, and workload.

AI ERP relevance:

The AI ERP extension is an applied design framework. Exact isolation, retention, model, and service choices must be validated against the company's products and regulations.

Companion guide:

Campfire\_Senior\_Data\_Engineer\_System\_Design\_QA.docx contains 28 detailed questions, Campfire-focused model answers, values guidance, story preparation, and follow-up prompts.